

HW10 — Chrome DevTools

Isaiah | CIT 484/L | CSUN

This document describes the Chrome DevTools implementations added to the HW10 demo page. Each section covers one required topic, explains the implementation, and includes screenshots taken from the live demo in Chrome.

1 Message Logging

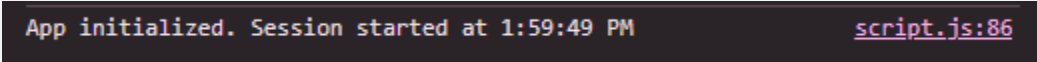
The Message Logging tab demonstrates the six primary console output methods. Each button on the page fires the real console call and shows an inline preview. Open DevTools (F12) and switch to the Console tab to see the live output.

1.1 Log Info — `console.info()`

`console.info()` outputs an informational message. DevTools renders it with a blue (i) icon and groups it under the Info filter level. It is functionally similar to `console.log()` but carries semantic meaning for status messages.

```
console.info('App initialized. Session started at ' + new Date().toLocaleTimeString());
```

Screenshot: Console tab showing the blue info entry after clicking Run console.info

A screenshot of a console log entry. The text is "App initialized. Session started at 1:59:49 PM" in a monospace font. To the right of the text is a link "script.js:86" in a purple color. The entire entry is on a dark background with a light border.

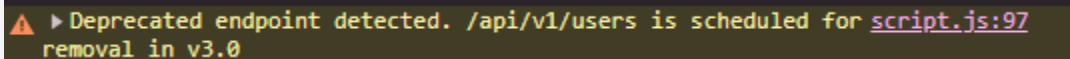
App initialized. Session started at 1:59:49 PM [script.js:86](#)

1.2 Log Warning — `console.warn()`

`console.warn()` outputs a warning message highlighted in yellow with a triangle icon. DevTools increments the warning counter in the toolbar. Use this for non-fatal issues like deprecated API usage or slow response times.

```
console.warn('Deprecated endpoint detected. /api/v1/users is scheduled for removal in v3.0');
```

Screenshot: Console tab showing the yellow warning entry after clicking Run console.warn

A screenshot of a console log entry. It starts with a yellow triangle warning icon. The text is "Deprecated endpoint detected. /api/v1/users is scheduled for removal in v3.0" in a monospace font. To the right of the text is a link "script.js:97" in a purple color. The entire entry is on a dark background with a light border.

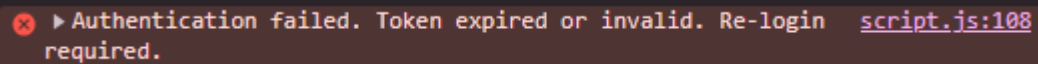
⚠ ▶ Deprecated endpoint detected. /api/v1/users is scheduled for [script.js:97](#) removal in v3.0

1.3 Log Error — `console.error()`

`console.error()` outputs an error message in red with an expandable stack trace attached. It increments the red error counter in the DevTools toolbar. Use it for failures, caught exceptions, and authentication errors.

```
console.error('Authentication failed. Token expired or invalid.');
```

Screenshot: Console tab showing the red error entry with stack trace



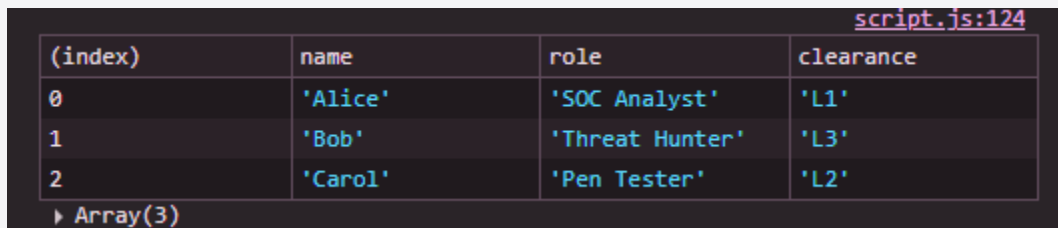
✖ ▶ Authentication failed. Token expired or invalid. Re-login [script.js:108](#) required.

1.4 Log Table — console.table()

`console.table()` accepts an array of objects and renders them as a sortable, interactive table inside DevTools. Each object key becomes a column header. This is useful for inspecting structured data like API responses or alert lists.

```
var analysts = [
  { name: 'Alice', role: 'SOC Analyst', clearance: 'L1' },
  { name: 'Bob', role: 'Threat Hunter', clearance: 'L3' },
  { name: 'Carol', role: 'Pen Tester', clearance: 'L2' },
];
console.table(analysts);
```

Screenshot: Console tab showing the sortable table rendered by `console.table()`



(index)	name	role	clearance
0	'Alice'	'SOC Analyst'	'L1'
1	'Bob'	'Threat Hunter'	'L3'
2	'Carol'	'Pen Tester'	'L2'

▶ Array(3)

1.5 Log Group — console.group()

`console.group()` opens a collapsible group header in DevTools. All console calls between `group()` and `groupEnd()` are nested under that header. Groups can be nested. This demo nests a Response Headers group inside a Network Request Details group.

```
console.group('Network Request Details');
console.log('URL: https://api.example.com/v2/threats');
console.info('Status: 200 OK');
console.group('Response Headers');
console.log('Content-Type: application/json');
console.groupEnd();
console.warn('Response time: 843ms above threshold');
console.groupEnd();
```

Screenshot: Console tab showing the nested collapsible group structure

▼ Network Request Details	script.js:135
URL: https://api.example.com/v2/threats	script.js:136
Method: GET	script.js:137
Status: 200 OK	script.js:138
▼ Response Headers	script.js:139
Content-Type: application/json	script.js:140
X-RateLimit-Remaining: 95	script.js:141
⚠ ▶ Response time: 843ms above threshold	script.js:143

1.6 Log Custom — console.log() with %c

The %c format specifier applies inline CSS to console output. The style string is passed as the second argument. Multiple %c tokens can appear in one call. This feature is Chromium-only and is useful for building custom debug banners.

```
var s1 = 'background:#0a1a0f;color:#3dffa0;font-weight:600;padding:4px 10px';
var s2 = 'background:#0b1e2e;color:#4db8ff;font-size:11px;padding:3px 10px';
console.log('%c[ALERT] Intrusion detected', s1);
console.log('%cSource IP: 192.168.4.22 Severity: HIGH', s2);
```

Screenshot: Console tab showing the CSS-styled log output

[ALERT] Intrusion detected	script.js:167
Source IP: 192.168.4.22 Severity: HIGH	script.js:168
Timestamp: 2026-05-05T21:00:51.078Z	script.js:169

2 View Messages Logged by the Browser

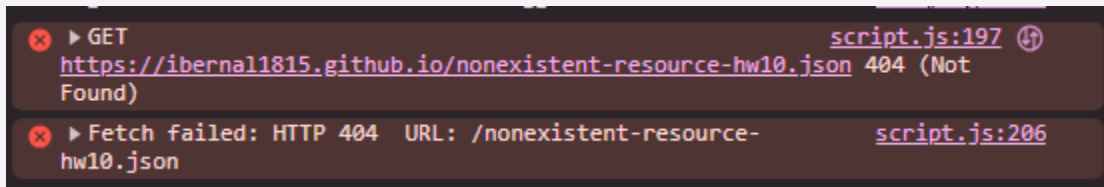
This section demonstrates the three types of messages the browser logs automatically when something goes wrong at runtime. These are not written by the developer — the browser generates them in response to network failures, JavaScript errors, and performance violations.

2.1 Cause a 404 Network Error

Calling `fetch()` with a path that does not exist on the server returns HTTP 404. The browser logs a red network error entry in the Console automatically. The same request also appears in the Network tab with status 404. Filter by Fetch or XHR to isolate it.

```
fetch('/nonexistent-resource-hw10.json')
.then(function(r) {
  if (!r.ok) throw new Error('HTTP ' + r.status + ' ' + r.statusText);
  return r.json();
})
.catch(function(err) {
  console.error('Fetch failed:', err.message);
});
```

Screenshot: Console tab showing the red 404 network error entry



Screenshot: Network tab showing the failed request with status 404

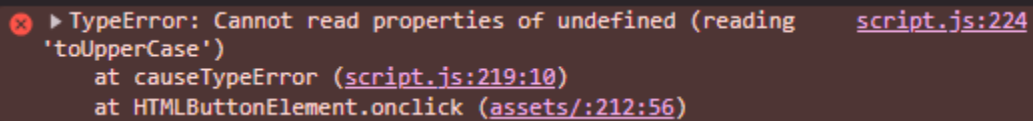
✖ nonexistent-resource-hw10.json	404	fetch	script.js:197	5.6 kB	130 ms
----------------------------------	-----	-------	-------------------------------	--------	--------

2.2 Cause a TypeError

Calling a method on undefined throws a `TypeError` at runtime. The browser logs it in red with a full stack trace pointing to the source line. In this demo the error is caught with `try/catch` and re-logged with `console.error()` so the stack trace is visible without crashing the page.

```
try {
  var data = undefined;
  data.toUpperCase(); // TypeError: Cannot read properties of undefined
} catch(e) {
  console.error(e);
}
```

Screenshot: Console tab showing the red `TypeError` entry with stack trace



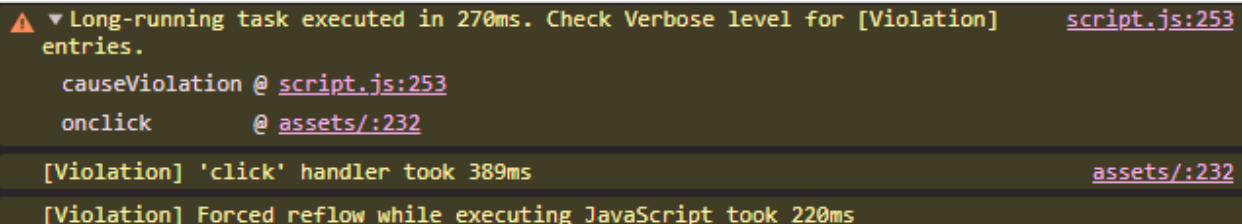
```
✖ ▶ TypeError: Cannot read properties of undefined (reading 'toUpperCase')  
    at causeTypeError (script.js:219:10)  
    at HTMLButtonElement.onclick (assets/:212:56)
```

2.3 Cause a Violation

Chrome generates [Violation] warnings when the main thread is blocked longer than ~50ms. Two patterns trigger them: forced layout reflow (alternating DOM style writes and `offsetWidth` reads in a loop) and busy-wait loops. Set the Console log level to Verbose to see these entries — they are hidden at the Default level.

```
// forced reflow: alternates write/read 20,000 times  
for (var i = 0; i < 20000; i++) {  
  temp.style.width = (i % 200) + 'px'; // write - marks layout dirty  
  void temp.offsetWidth; // read - forces synchronous reflow  
}  
  
// busy-wait: blocks the main thread for ~120ms  
var now = Date.now();  
while (Date.now() - now < 120) {}
```

Screenshot: Console set to Verbose level showing yellow [Violation] entries



```
⚠ ▼ Long-running task executed in 270ms. Check Verbose level for [Violation] entries. script.js:253  
  causeViolation @ script.js:253  
  onclick @ assets/:232  
[Violation] 'click' handler took 389ms assets/:232  
[Violation] Forced reflow while executing JavaScript took 220ms
```

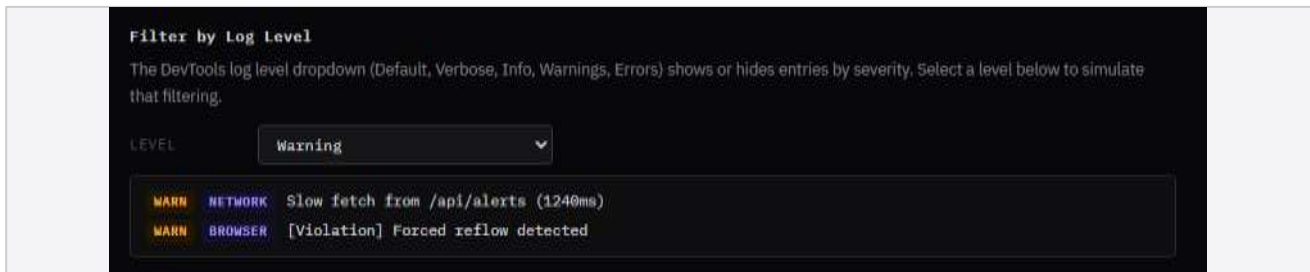
3 Filter Messages

The Filter Messages tab seeds the DevTools Console with 10 mixed entries across different levels (info, warn, error, log) and sources (script, network, browser). Each filter control below mirrors the corresponding feature in the DevTools filter bar.

3.1 Filter by Log Level

The log-level dropdown in the top-right of the Console panel (Default, Verbose, Info, Warnings, Errors) shows or hides entries by severity. Selecting Warnings hides all info and log entries, leaving only warnings and errors visible.

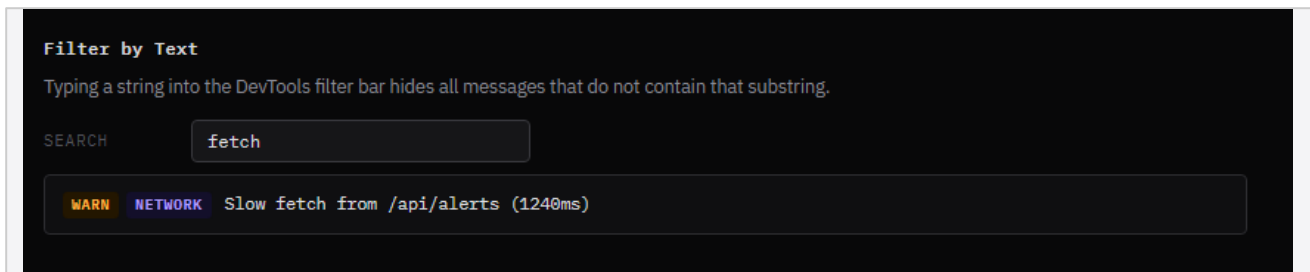
Screenshot: DevTools Console with log level set to Warnings — only warn/error entries visible



3.2 Filter by Text

Typing a plain-text string into the filter bar at the top of the Console hides all messages that do not contain that substring. The match is case insensitive. Typing 'fetch' shows only entries whose message contains the word fetch.

Screenshot: Console filter bar with 'fetch' typed — only matching entries visible



3.3 Filter by Regular Expression

Prefixing the filter bar input with / enables regex mode in DevTools. Standard JS regex syntax is supported including flags. The pattern /404|type/i matches messages containing either '404' or 'type' regardless of case.

```
/404|type/i
```

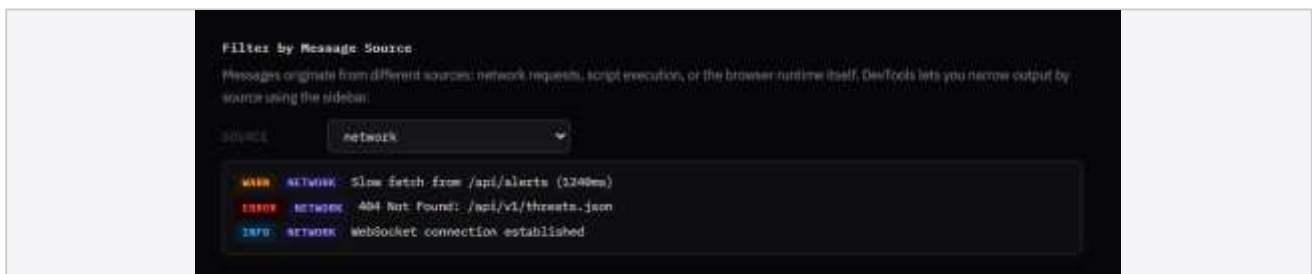
Screenshot: Console filter bar with regex pattern — only matching entries visible



3.4 Filter by Message Source

DevTools categorizes console messages by origin. The sidebar lets you filter by source: network (fetch and XHR errors), script (developer-written console calls), or browser (CSP violations, deprecation notices, and other runtime messages).

Screenshot: Console sidebar with 'network' source selected — only network-origin entries visible



3.5 Filter by User Messages

The User Messages shortcut in the DevTools Console sidebar shows only messages written by the developer's own JavaScript code via console methods. It hides everything the browser logged automatically, making it easier to focus on your own debug output.

Screenshot: Console with User Messages filter active — only developer-written entries visible



4 Debugging

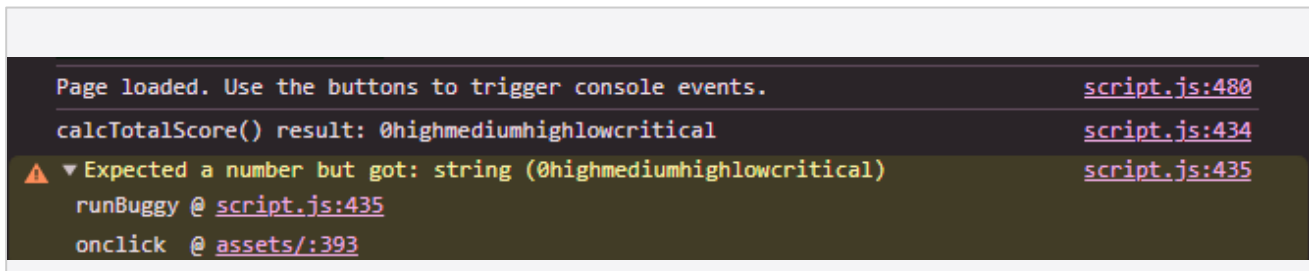
The Debugging tab walks through a complete debugging workflow using a realistic bug. The scenario: a security dashboard function `calcTotalScore()` is supposed to sum numeric risk scores from a list of alerts and return the total. Instead it returns `NaN` because it reads `alert.severity` (a string like 'high') instead of `alert.score` (a number).

4.1 Reproduce the Bug

Clicking 'Run Buggy Function' calls `calcTotalScore()` and logs the result to the Console. The expected total is 349 (85+52+91+23+98). Instead `NaN` is returned because `alert.severity` is a string — adding a string to a number in JavaScript produces `NaN`, and `NaN` added to anything stays `NaN`.

```
function calcTotalScore() {  
  var total = 0;  
  alerts.forEach(function(alert) {  
    total += alert.severity; // BUG: severity is a string, not a number  
  });  
  return total;  
}
```

Screenshot: Console showing NaN logged as the result of calcTotalScore()



4.2 Get Familiar with the Sources UI

The Sources panel has three panes. The File Navigator (left) lists every file the page loaded. The Code Editor (center) shows the source code of the selected file — click any line number to set a breakpoint. The Debugger pane (right) shows Breakpoints, Scope variables, Watch Expressions, and the Call Stack when execution is paused.

Screenshot: Sources panel open with script.js selected — all three panes visible

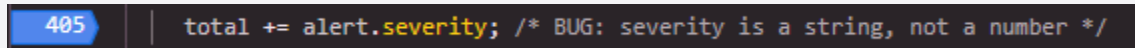


4.3 Set a Line-of-Code Breakpoint

In the Sources panel Code Editor with script.js open, scroll to calcTotalScore() and click the line number next to the line that reads `total += alert.severity`. A blue marker appears on that line number confirming the breakpoint is active. The same entry appears in the Breakpoints list in the Debugger pane on the right.

```
total += alert.severity; // click this line number in Sources
```

Screenshot: Blue breakpoint marker on the line, and the Breakpoints list entry in the right pane

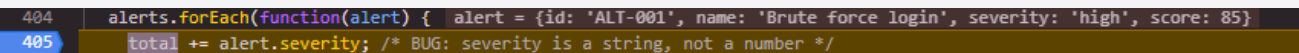


```
405 | total += alert.severity; /* BUG: severity is a string, not a number */
```

4.4 Pause the Code with a Breakpoint

With the breakpoint set, clicking 'Run and Hit Breakpoint' calls calcTotalScore() again. DevTools intercepts execution on the breakpoint line before it runs. The current line is highlighted in blue in the Code Editor. The page pauses until you press the play button (F8) to resume.

Screenshot: Sources panel paused on the highlighted breakpoint line with the blue execution marker



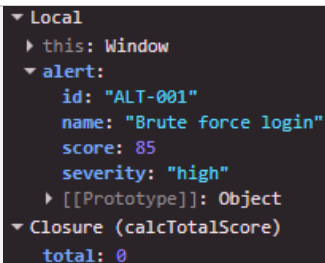
```
404 | alerts.forEach(function(alert) { alert = {id: 'ALT-001', name: 'Brute force login', severity: 'high', score: 85}
405 | total += alert.severity; /* BUG: severity is a string, not a number */
```

4.5 Check Variable Values

Scope Pane

While paused, the Scope section in the Debugger pane shows every variable in scope at the current line. Expanding the local scope reveals `total`, `alert`, and the full `alert` object. `alert.severity` shows a string value like 'high' — confirming it is not a number and cannot be summed.

Screenshot: Scope section expanded showing 'alert' object with severity as a string value

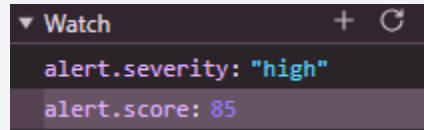


```
▼ Local
  ▶ this: Window
  ▼ alert:
    id: "ALT-001"
    name: "Brute force login"
    score: 85
    severity: "high"
    ▶ [[Prototype]]: Object
  ▼ Closure (calcTotalScore)
    total: 0
```

Watch Expressions

In the Watch section of the Debugger pane, click the + button and add two expressions: `alert.severity` and `alert.score`. DevTools evaluates both in the current scope. `alert.severity` returns a string and `alert.score` returns a number — making the bug immediately clear side by side.

Screenshot: Watch section showing `alert.severity` (string) and `alert.score` (number) evaluated

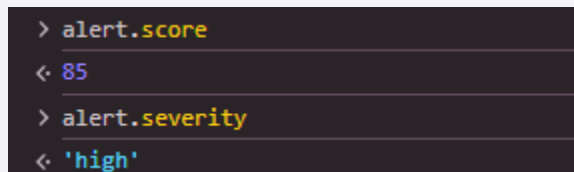


The Console

While still paused, switch to the Console tab. Expressions typed here evaluate in the current paused scope. Typing `alert.score` returns the numeric value. Typing `typeof alert.severity` returns `'string'` — confirming the root cause of the bug.

```
alert.score // returns a number, e.g. 85
typeof alert.severity // returns 'string'
```

Screenshot: Console while paused showing `alert.score` and `typeof alert.severity` evaluated



4.6 Apply the Fix

Resume execution by pressing F8 or the play button, then remove the breakpoint by clicking the blue marker again. The fix is to change `alert.severity` to `alert.score` inside the `forEach` loop. The corrected function `calcTotalScoreFixed()` is already in `script.js`. Clicking 'Run Fixed Function' confirms it returns 349 and logs a breakdown table to the Console.

Before (buggy)	After (fixed)
<code>total += alert.severity;</code>	<code>total += alert.score;</code>

Screenshot: Output box and Console showing 349 as the correct total from `calcTotalScoreFixed()`

